Category:

Reversing

Name:

Packer

Message:

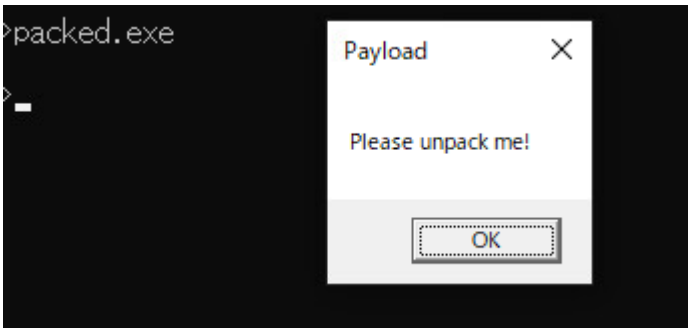
Analyze the given binary and retrieve the hidden flag.

Extraction password is “csg2025”.

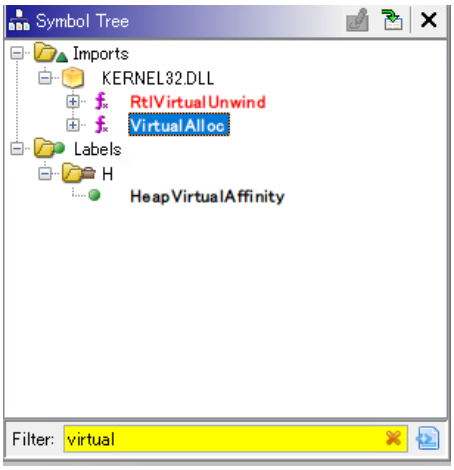
To proof your understanding, please create flag by “**CSG_FLAG{packer_daisuki}**”

Instructions:

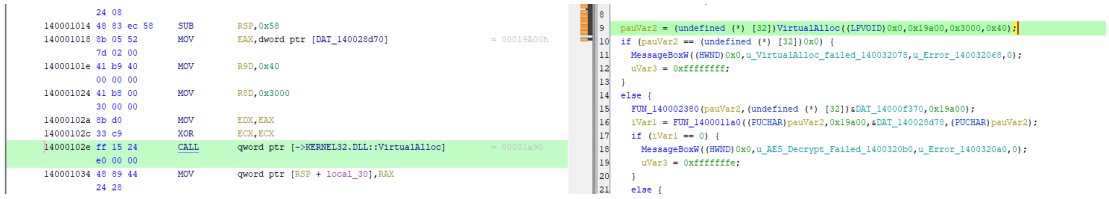
1. When the file is executed, a message box appears. Since it is likely packed, perform reverse engineering to determine how it can be unpacked.



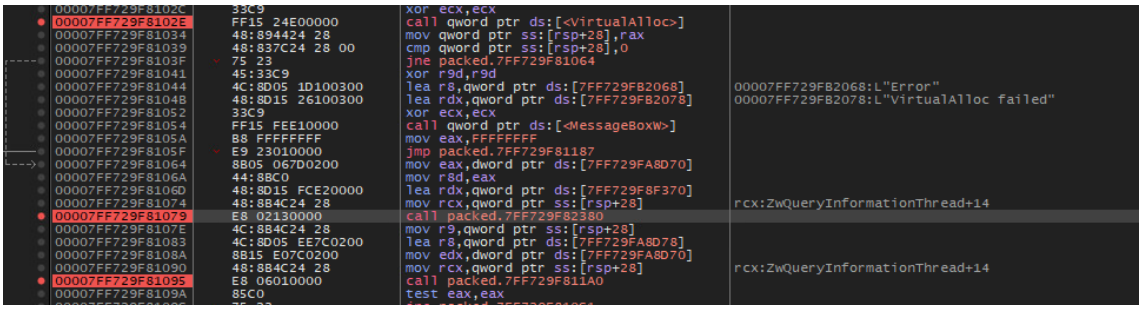
2. A packer generally consists of two main parts: the **stub**, which implements the self-unpacking logic, and the encrypted **blob**. During the self-unpacking process, the blob (which is the encrypted payload) is typically decrypted and then loaded into memory. Therefore, around the point where the **VirtualAlloc** function is called, it may be possible to locate the unpacked payload that the packer has already decrypted and dump it from memory.
   Load the binary into a reverse engineering tool such as Ghidra, identify the location where the VirtualAlloc function is called from the Symbol Tree.



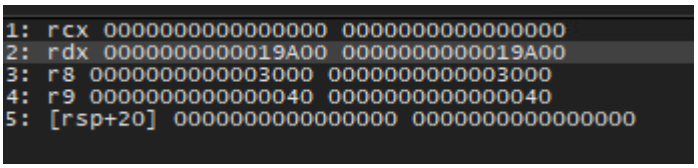
The VirtualAlloc call and a function that appears to handle decryption can be identified. This sequence suggests a typical flow in which the encrypted payload (the blob) is decrypted and the decrypted data is then written into the allocated memory.

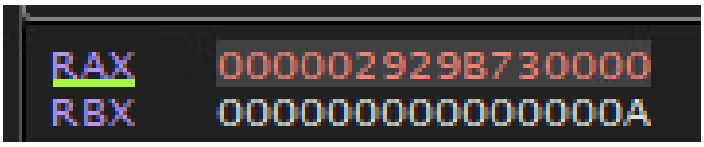


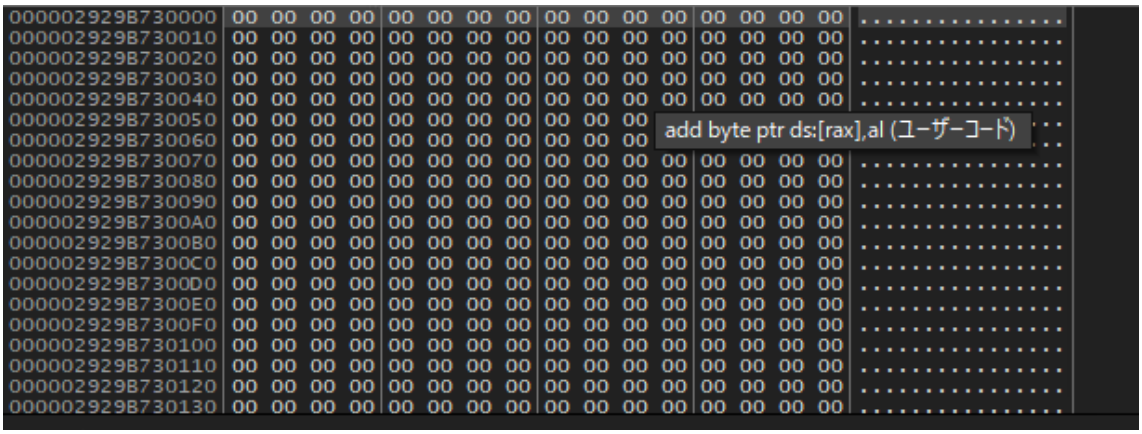
3. Set breakpoints at the RVAs **0x102E**, **0x1079**, and **0x1095** in x64dbg. These correspond to the RVAs of the functions located around the earlier VirtualAlloc call.



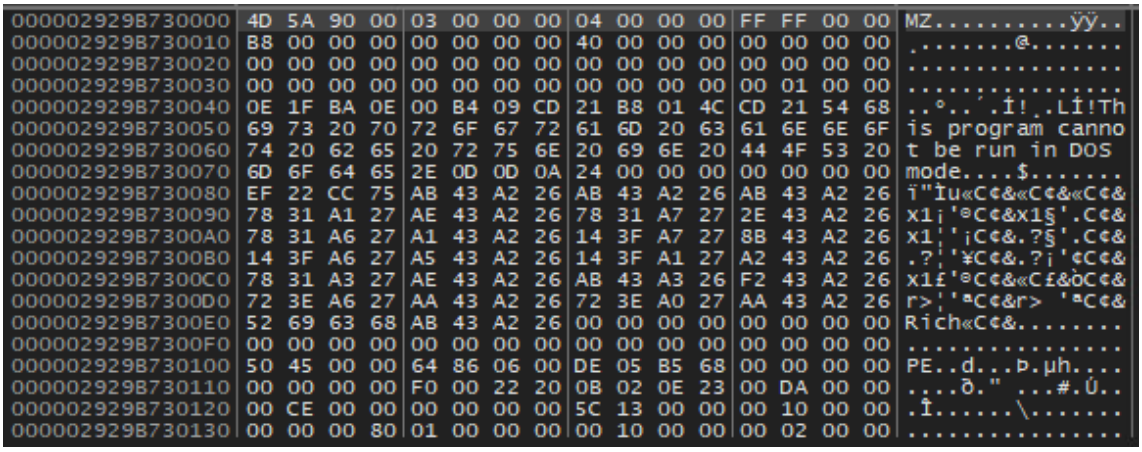
4. At this point, check Microsoft's documentation to review the details of the VirtualAlloc function. According to the documentation, the first argument specifies the starting address of the memory to be allocated, the second argument specifies the size of the memory, and the return value is the base address of the allocated memory region.
   [VirtualAlloc function (memoryapi.h) - Win32 apps | Microsoft Learn]
5. At the time the VirtualAlloc function is executed, the value of the **second argument is stored in RDX. This value is 19A00**, which represents the size. **After the function call, the return value is stored in RAX**, **which holds the base address of the allocated memory.** Since the decrypted payload is expected to be unpacked into this memory address after several subsequent operations, need to monitor this address.







6. While stepping through the debugger, it becomes clear that after executing the function located at RVA 0x1095, data is unpacked into the memory address allocated by VirtualAlloc.

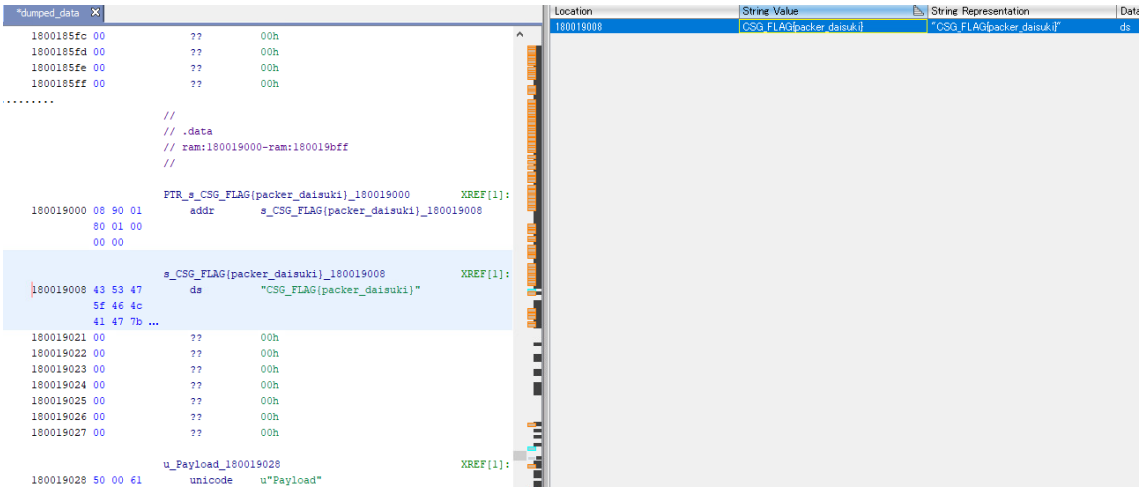


7. From the arguments of the earlier VirtualAlloc call, we know that the data size is 19A00. Since MZ is the magic number of a binary, dump 19A00 bytes starting from the address where this magic number begins. In x64dbg, the following command can be used to dump data of any specified size:

   savedata <file name>, <address>, <size>



8. Importing the dumped data into Ghidra reveals a hardcoded flag through the Defined Strings feature.



Flag: **CSG_FLAG{packer_daisuki}**